

Homework



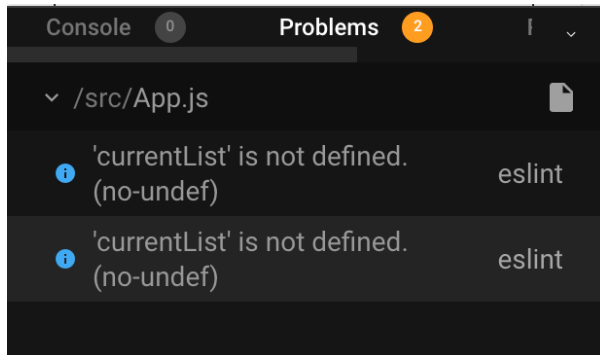
Homework

- In today's homework, you will continue on the **ToDo List** app using React.
- You will continue use **Code Sandbox** for this homework.
- Please fork the app (from last homework) in Sandbox before you begin this homework. Modify the name to include **v4** (as in version 4).
- In the last homework, you'd:
 - Add a controlled form to allow user add new todo items to the current display list.
 - Use `setState` to update the state data.
 - Add buttons to each todo item to mark/toggle items that are completed.
 - Used more Reactstrap, Bootstrap and Font Awesome to format and style JSX elements.



Homework

- In this homework, you will:
 - Move data and child components to its own external file/component.
 - Create a new component to be linked via React Router.
- Before we begin, let's address a couple of minor issues from the last homework:



Add this
to resolve
the issue



```
);  
this.currentList = this.state.TODOList1;  
} else if (n === "Tuesday") {  
  todo = (  
    <Todos  
      todos={this.state.TODOList2}  
      updateTodo={this.handleToggleTodo}  
    />  
  );  
  this.currentList = this.state.TODOList2;  
}
```



Homework

- Moving along...we'll start with the first task – ie. moving the state data externally to its own component file. The advantage of this move is that data can be maintain separately in the long haul.
- Go ahead and create a new js file in the src folder, save as **TodoData.js**.
- Copy **this.state = { }** in **App.js** and paste into **TodoData.js**.
- Inside **TodoData.js**, modify the first line **this.state = {** to **export const TodoData = {**.
- Returning to **App.js** – import the data file and remove everything within **this.state = {...}**:

```
import React, { Component } from "react";  
import { TodoData } from "../TodoData";  
import { ListGroup, ListGroupItem } from "reactstrap";
```

```
super(props);  
this.state = {  
  TodoLists: TodoData  
};
```



Homework

- The data is connected to App.js but because the state now has a property name **TodoLists**, all subsequent reference to the data must use this property name first.
- Within the conditional statement, modify the state data reference as follow:

```
todo = (  
  <Todos  
    todolists={this.state.TodoLists.TodoList1}  
    updateTodo={this.handleToggleTodo}  
  />  
);  
this.currentList = this.state.TodoLists.TodoList1;
```

```
todo = (  
  <Todos  
    todolists={this.state.TodoLists.TodoList1}  
    updateTodo={this.handleToggleTodo}  
  />  
);  
this.currentList = this.state.TodoLists.TodoList1;
```



Homework

- Next, we will move functional components out to its own component file as well. We'll start with moving **Todos** and **AddTodo**.
- Create a new js file in the src folder, name it as **Todos.js**. Cut the **Todos function** from App.js and paste it in **Todos.js**, starting at line 3.
- We also need to export this function so other component files have access to it, such as App.js. At the bottom of **Todos.js**, add this export script:

```
    </ListGroup>  
  );  
}  
  
export default Todos;
```



Homework

- Notice the imports in App.js:

```
import { ListGroup, ListGroupItem } from "reactstrap";  
import "bootstrap/dist/css/bootstrap.min.css";  
import $ from "jQuery";  
import { faCheck, faTimes } from "@fortawesome/free-solid-svg-icons"  
import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
```

- Any imports not use will appear dim out. We need to copy these 4 imports onto Todos.js.

```
1 import { ListGroup, ListGroupItem } from "reactstrap";  
2 import $ from "jQuery";  
3 import { faCheck, faTimes } from "@fortawesome/free-solid-svg-icons"  
4 import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";  
5  
6 function Todos(props) {
```



Homework

- Like the the Todos function, the [AddTodo function](#) is also called from App.js and in order to maintain that, we need to import this in App.js:

```
import Todos from "../Todos";  
import AddTodo from "../AddTodo";  
import { ListGroup, ListGroupItem } from "reactstrap";
```

- Things should return back to normal and the data is displaying once again.
- Before we separate one last part of the script from App.js to its own component, let's discuss what we want to achieve in the next task. We want to create a navigation bar at the top of this app that will contain 2 buttons – 1) [Home](#) 2) [All Todo Lists](#). Clicking on the [home](#) button will display the content of the home (which is what our current app is showing). Clicking on the [All Todo Lists](#) button will display the entire list of todos.



Homework

- The Todos function is being called from App.js and in order to maintain that, we need to import the Todos function in App.js:

```
import { TodoData } from "../TodoData";  
import Todos from "../Todos";  
import { ListGroup, ListGroupItem } from "reactstrap";
```

- Next, create a new js file in the src folder and save it as **AddTodo.js**. Cut the AddTodo function in App.js and paste it in **AddTodo.js**, on line 3. Then, import the following at the top while at the bottom of the document, add an export:

```
1 import React from "react"  
2  
3 function AddTodo(props) {
```

```
    </div>  
    );  
}  
  
export default AddTodo;
```



Homework

- Okay...so let's start by creating our home component. Currently our home content is all part of the App class. We will separate it. The App class will maintain its role – that is render the final view. It will render the new nav component and any other components that we feel necessary later.
- Create a new js file in the src folder and save it as **Home.js**. In App.js, copy everything within the App class...ie. including **constructor {.....}** and the rest, and paste into **Home.js**, on line 4.
- Then, create a class called **Home** that wraps the entire block of code that you'd pasted:

```
3  class Home extends Component {  
4    constructor(props) {
```



Homework

- With the home content separated, we need to export the component and also import the necessary library components and file exports:

```
    }  
  }  
  
  export default Home;
```

```
1  import React, { Component } from "react";  
2  import { TodoData } from "../TodoData";  
3  import AddTodo from "../AddTodo";  
4  import Todos from "../Todos";  
5  
6  class Home extends Component {
```

- Now we can return to App.js and start removing parts of the codes that is no longer needed.



Homework

- In App.js, remove
 1. the 2 handlers between the constructor and render().
 2. from below render() ie. `let n = .....` up until before return ie. `end of the if statement`.
 3. all codes within the div element (excluding div) in the return.
 4. all unused imports. These are all the remaining ones needed in the App component:

```
1  import "./styles.css";
2  import React, { Component } from "react";
3  import { TodoData } from "./TodoData";
4  import "bootstrap/dist/css/bootstrap.min.css";
```



Homework

- And this is what you have left within render() :

```
render() {  
  return (  
    {  
      /* Output final view */  
    },  
    (  
      <div className="App text-secondary">  
  
        </div>  
    )  
  );  
}
```



```
export default App;
```

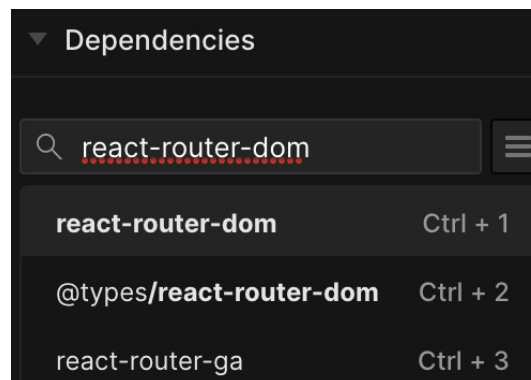


Homework

- Next, we'll create the new navigation that contains the 2 buttons we'd discussed earlier.
- Create a new js file in the src folder and save it as **Nav.js**. In this Nav.js file, let's start with a function name Nav on line 3:

```
3  function Nav() {  
4  
5  }
```

- Before we continue, let's add the react router dom dependencies:





Homework

- We'll import the necessary components from the react router dom and a couple of icons from font awesome:

```
1 import { Link, BrowserRouter as Router, Route, Switch } from "react-router-dom";
2 import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
3 import { faClipboardList, faHome } from "@fortawesome/free-solid-svg-icons";
```

- In the Nav function, add a return and within it, a jsx element as wrapper and within it, a router:

```
function Nav() {
  return (
    <div>
      <Router>

      </Router>
    </div>
  )
}
```



Homework

- Within the router, first let's set up the navigation elements:

```
<Router>
  { /* Navigation */ }
  <ul>
    <li></li>
    <li></li>
  </ul>
</Router>
```

- And for each list element, add a **link** with their respective **to** path:

```
<li>
  <Link to="/" />
</li>
<li>
  <Link to="/allLists" />
</li>
```




Homework

- And for each link, we'll add a font-awesome icon:

```
<Link to="/">
  <FontAwesomeIcon
    icon={faHome}
    className="fas fa-2x my-3 mr-2 text-white"
  />
</Link>
```

```
<Link to="/allLists">
  <FontAwesomeIcon
    icon={faClipboardList}
    className="fas fa-2x my-3 text-white"
  />
</Link>
```



Homework

- Below the navigation (still within the router), we will create the routes using a switch (if-elseif-else) statement to find the matching routes:

```
</ul>

{/* Routes */}
<Switch>
  <Route exact path="/">
  </Route>
  <Route path="/allLists">
  </Route>
</Switch>
</Router>
```



Homework

- For each route, these are the components they will call respectively:

```
<Route exact path="/">
  <Home />
</Route>
<Route path="/allLists">
  <AllTodos />
</Route>
```

- Note:
 - the `Home` component already exist but need to be imported here for access.
 - the `AllTodos` component is not exist yet, but also need to be imported here for access.



Homework

- Before we forget, we need to export the [Nav](#) component:

```
    </div>
  )
}

export default Nav;
```

- And import the [Home](#) and [AllTodos](#) components:

```
3 import { faClipboardList, faHome } from
4 import Home from "../Home";
5 import AllTodos from "../AllTodos";
```



Homework

- Next, create a new js file in the src folder and save it as **AllTodos.js**. In this file, we'll create a function name AllTodos on line 1 and within it, a return and within it, a jsx element as wrapper containing a heading:

```
export default function AllTodos() {  
  return (  
    <div>  
      <h2>Todo Lists</h2>  
    </div>  
  )  
}
```

Note:

Export can be added on the component instead of at the bottom of the document.



Homework

- Below the heading, we will create 2 jsx elements with its respective sub heading:

```
return (  
  <div>  
    <h2>Todo Lists</h2>  
    <div className="border mx-5 my-3 py-3">  
      <h3 className="text-info">Monday</h3>  
    </div>  
    <div className="border mx-5 my-3 py-3">  
      <h3 className="text-info">Tuesday</h3>  
    </div>  
  </div>  
)
```



Homework

- So this how this AllTodos component works:

App → Nav → AllTodos

- Data from the App will pass to the Nav and then pass to AllTodos.
- Remember earlier I'd mentioned that App will render the final view output. The app will always display the nav but not the home and alltodos because these two will display based on button clicks on the nav. Home however will display by default. So therefore, in the App class, we will only need to render the nav component. But while calling this component, we will pass data to it. But first let's import the nav component in App.js so we can render it:

```
3 import { TodoData } from "./TodoData";  
4 import Nav from "./Nav";  
5 import "bootstrap/dist/css/bootstrap.min.css";
```



Homework

- Now we can proceed to render/call the nav component and pass data to it:

```
<div className="App text-secondary">  
  <Nav  
    lists1={this.state.TODOLists.TODOList1}  
    lists2={this.state.TODOLists.TODOList2}  
  />  
</div>
```

- On Nav.js, we will now add props as parameter in the component to receive the data from App.js.

```
function Nav(props) {  
  return (  

```




Homework

- In the AllTodos route, we can now pass along the data received as props in the calling of the component:

```
<Route path="/allLists">  
  <AllTodos lists1={props.lists1} lists2={props.lists2} />  
</Route>
```

- We can now proceed to the AllTodos.js component itself to receive the data from the router, when the user clicks the button on the nav. First, let's insert the props parameter:

```
export default function AllTodos(props) {  
  return (  

```



Homework

- Next, we'll use the array map method to map out the text of each list to jsx elements:

```
<div className="border mx-5 my-3 py-3">
  <h3 className="text-info">Monday</h3>
  {props.lists1.map((list1) => {
    return <div>{list1.text}</div>;
  })}
</div>
```

```
<div className="border mx-5 my-3 py-3">
  <h3 className="text-info">Tuesday</h3>
  {props.lists2.map((list2) => {
    return <div>{list2.text}</div>;
  })}
</div>
```



Homework

- Technically we are done. However, on our browser view, the navigation doesn't seem like it's displaying, at least not correctly:



- We will add some classes to adjust this. Return to Nav.js:

```
function Nav(props) {  
  return (  
    <div className="header bg-dark">  
      <Router>
```



Homework

- And the following class:

```
<Router>
  { /* Navigation */}
  <ul className="nav mx-2 mb-4">
    <li>
```

- And finally, a custom css style to adjust the height. In [style.css](#):

```
.header {
  height: 70px;
}
```



Homework

- Testing the app on the browser view:

A screenshot of a web application interface. At the top is a dark header with a home icon and a clipboard icon. Below the header, the title 'Monday's Todo List' is displayed in large, bold text, with 'Monday's' in orange and 'Todo List' in grey. Underneath the title is a list of three tasks, each in a teal box with a green checkmark to its right: 'Eat Breakfast', 'Walk Dog', and 'Take Shower'. At the bottom, there is a white box containing two input fields: 'Add Todo Item' and 'Add Todo Note', followed by a small 'Add' button.

A screenshot of a web application interface. At the top is a dark header with a home icon and a clipboard icon. Below the header, the title 'Todo Lists' is displayed in grey. Underneath the title are two white boxes, each representing a day's todo list. The first box is titled 'Monday' in teal and contains the tasks 'Eat Breakfast', 'Walk Dog', and 'Take Shower'. The second box is titled 'Tuesday' in teal and contains the tasks 'Get Lunch', 'Run a Mile', and 'Take Bath'.



Homework

- **DUE:**

On or before this Sunday, 10.30PM PT.

- **Submission:**

Post Code Sandbox URL on GAP Week 2 Day 2 Homework



Connect with Us (WEB403/603/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

Teaching
Assistant

Erin Tustin

erintustin@westcliff.edu

End of Presentation